

Rendu TP Design Pattern (21/05/2026)

Anton MAIGNAN – ESGI 4

main.cpp

```
#include <iostream>

#include "Explosion.cpp"
#include "ExplosionFactory.cpp"
#include "ExplosionCacheProxy.cpp"

using namespace std;

int main()
{
    ExplosionCacheProxy proxy;
    ExplosionBuilder builder;
    ExplosionFactory factory;

    shared_ptr<ExplosionConfig> config1 = builder
        .withPosition(10, 10)
        .withColor("red")
        .withCount(5)
        .withSpread(10.0f)
        .build();
    Explosion exp1 = proxy.create(*config1);
    exp1.render();

    shared_ptr<ExplosionConfig> config2 = builder
        .withPosition(10, 10)
        .withColor("red")
        .withCount(5)
        .withSpread(10.0f)
        .build();
    Explosion exp2 = proxy.create(*config2);
    exp2.render();

    shared_ptr<ExplosionConfig> config3 = builder
        .withPosition(20, 20)
        .withColor("yellow")
        .withCount(10)
        .withSpread(10.0f)
        .build();
    Explosion exp3 = factory.create(*config3);
    exp3.render();

    return 0;
```

```
}
```

ElyweightFactory.cpp

```
// =====  
// ParticleFlyweight  
// Base shape (texture, shader, physics) partagée entre les particules du même typ  
// =====  
class ParticleFlyweight  
{  
private:  
    string texture;  
    string shader;  
    string physics;  
  
public:  
    ParticleFlyweight(const string &particuleType)  
    {  
        // Devrait avoir un fichier JSON répertoire pour récupérer  
        // les données de chaque type de particule. (ici on simule juste avec des s  
        this->texture = "texture_" + particuleType;  
        this->shader = "shader_" + particuleType;  
        this->physics = "physics_" + particuleType;  
    }  
  
    // GETTERS  
    string getTexture()  
    {  
        return texture;  
    }  
  
    string getShader()  
    {  
        return shader;  
    }  
  
    string getPhysics()  
    {  
        return physics;  
    }  
};  
  
// =====  
// FlyweightFactory  
// Méthodes :  
// - getInstance() : Retourne l'instance singleton de la factory.  
// - getParticle(const string &particuleType) : Retourne une instance de ParticleFl  
// =====  
class FlyweightFactory  
{  
private:  
    FlyweightFactory() {}
```

```

static FlyweightFactory* instance;

// Mutex pour protéger l'accès à l'instance singleton
static mutex factoryMutex;

// Map pour stocker les instances de ParticleFlyweight partagées
unordered_map<string, shared_ptr<ParticleFlyweight>> Particles;

~FlyweightFactory() {}

public:
static FlyweightFactory* getInstance()
{
    lock_guard<mutex> lock(factoryMutex);

    if (instance == nullptr) {
        instance = new FlyweightFactory();
    }

    return instance;
}

ParticleFlyweight* getParticle(const string &particleType)
{
    if (Particles.find(particleType) == Particles.end())
    {
        Particles[particleType] = make_shared<ParticleFlyweight>(particleType);
    }

    return Particles[particleType].get();
}
};

FlyweightFactory* FlyweightFactory::instance = nullptr;
mutex FlyweightFactory::factoryMutex;

```

ParticlePrototype.cpp

```

#pragma once
#include <iostream>

#include "ParticuleDecorator.cpp"

using namespace std;

// =====
// IParticlePrototype
// interface de clonage pour les particules.
// =====
class IParticlePrototype
{

```

```

{
public:
    virtual IParticlePrototype* clone() const = 0;

    virtual ~IParticlePrototype() {}
};

// =====
// ParticlePrototype
// Particule concrète représentant une particule
// =====
class ParticlePrototype : public IParticlePrototype, public IParticle
{
private:
    float size;
    string color;
    float speed;
    float x, y;
    string texture;
    string shader;
    string physics;

public:
    ParticlePrototype(float size, const string &color, float speed, float x, float y)
    {
        this->size = size;
        this->color = color;
        this->speed = speed;
        this->x = x;
        this->y = y;
        this->texture = texture;
        this->shader = shader;
        this->physics = physics;
    }

    IParticlePrototype* clone() const override
    {
        return new ParticlePrototype(*this);
    }

    void render() override
    {
        cout << "Rendering particle at (" << x << ", " << y << ") with size " << size << " and color " << color << ", texture " << texture << ", shader " << shader << ", physics " << physics << endl;
    }
};

```

Explosion.cpp

```

#pragma once
#include <iostream>
using namespace std;

```

```

// =====
// ExplosionConfig - IMMUTABLE
// Configuration d'une explosion
// (position, couleur, nombre de particules, rayon d'explosion, durée de vie)
// =====
class ExplosionConfig
{
private:
    float x, y;
    string color;
    int particleCount;
    float spread;
    float lifetime;
    string type;

public:
    ExplosionConfig(float x, float y, const string &color, int particleCount, float
    {
        this->x = x;
        this->y = y;
        this->color = color;
        this->particleCount = particleCount;
        this->spread = spread;
        this->lifetime = lifetime;
        this->type = type;
    }

    // GETTERS
    float getX()
    {
        return x;
    }

    float getY()
    {
        return y;
    }

    string getColor()
    {
        return color;
    }

    int getParticleCount()
    {
        return particleCount;
    }

    float getSpread()
    {
        return spread;
    }
}

```

```

float getLifetime()
{
    return lifetime;
}

```

```

string getType()
{
    return type;
}
};

```

```

// =====
// ExplosionBuilder - BUILDER
// Méthodes :
// - withPosition(x,y)
// - withColor(Color c)
// - withCount(int n)
// - withSpread(float s)
// - withLifetime(float t)
// - build(): retourne un ExplosionConfig.
// =====

```

```

class ExplosionBuilder
{
private:
    float x, y;
    string color;
    int particleCount;
    float spread;
    float lifetime;
    string type;

```

```

public:
    ExplosionBuilder() { reset(); }
    void reset() {
        x = 0;
        y = 0;
        color = "no-color";
        particleCount = 1;
        spread = 0;
        lifetime = 0;
        type = "default";
    }

```

```

ExplosionBuilder& withPosition(float x, float y)
{
    this->x = x;
    this->y = y;
    return *this;
}

```

```

ExplosionBuilder& withColor(const string &color)
{

```

```

    this->color = color;
    return *this;
}

ExplosionBuilder& withCount(int count)
{
    this->particleCount = count;
    return *this;
}

ExplosionBuilder& withSpread(float spread)
{
    this->spread = spread;
    return *this;
}

ExplosionBuilder& withLifetime(float lifetime)
{
    this->lifetime = lifetime;
    return *this;
}

ExplosionBuilder& withType(const string &type)
{
    this->type = type;
    return *this;
}

shared_ptr<ExplosionConfig> build()
{
    shared_ptr<ExplosionConfig> result = make_shared<ExplosionConfig>(x, y, col
    reset();
    return result;
}
};

```

ExplosionFactory.cpp

```

#pragma once
#include <iostream>

#include "ParticlePrototype.cpp"
#include "ParticuleDecorator.cpp"
#include "FlyweightFactory.cpp"
#include "Explosion.cpp"

using namespace std;

// =====
// Explosion
// Contient une collection de particules créées à partir d'une configuration d'exp

```

```
// =====
class Explosion
{
private:
    vector<shared_ptr<IParticle>> particles;
public:
    void addParticle(shared_ptr<IParticle> particle)
    {
        particles.push_back(particle);
    }

    void render()
    {
        cout << "=== Rendering Explosion ===" << endl;
        cout << "[Explosion] Rendering explosion with " << particles.size() << " pa
        for (const auto& p : particles) {
            p->render();
            cout << "\n" << endl;
        }
        cout << "=====\n" << endl;

    }
};

// =====
// IExplosionFactory
// Interface pour la factory d'explosion
// =====
class IExplosionFactory
{
public:
    virtual Explosion create(ExplosionConfig &config) = 0;
    virtual ~IExplosionFactory() {}
};

// =====
// ExplosionFactory
// Factory pour créer des explosions à partir d'une configuration d'explosion
// =====
class ExplosionFactory: public IExplosionFactory
{
private:
    float randomSpeedFromSpread(float spread) const
    {
        return ((float)rand() / (float)RAND_MAX) * spread - (spread / 2);
    }

    float randomSizeFromSpread(float spread) const
    {
        return ((float)rand() / (float)RAND_MAX) * spread + 0.5f;
    }
};

```

```

IParticle* addRandomDecoration(IParticle* base) const
{
    int randVal = rand() % 4;

    switch (randVal)
    {
        case 0:
            return new GlowDecorator(base);
        case 1:
            return new ShadowDecorator(base);
        case 2:
            return new FadeOutDecorator(base);
        default:
            return base;
    }
}

public:
    ExplosionFactory() {}

    Explosion create(ExplosionConfig &config)
    {
        FlyweightFactory *flyweightFactory = FlyweightFactory::getInstance();
        Explosion explosion;

        ParticleFlyweight *prototype = flyweightFactory->getParticle(config.getType()

        ParticlePrototype *p = new ParticlePrototype(
            this->randomSizeFromSpread(config.getSpread()),
            config.getColor(),
            this->randomSpeedFromSpread(config.getSpread()),
            config.getX(),
            config.getY(),
            prototype->getTexture(),
            prototype->getShader(),
            prototype->getPhysics()
        );

        for (int i = 0; i < config.getParticleCount(); i++)
        {
            IParticlePrototype* clonedBase = p->clone();
            explosion.addParticle(shared_ptr<IParticle>(this->addRandomDecoration(d
        }

        delete p;

        return explosion;
    }
};

```

ExplosionCacheProxy.cpp

```

#pragma once
#include <iostream>

#include "ExplosionFactory.cpp"

using namespace std;

class ExplosionCacheProxy: public IExplosionFactory
{
private:
    IExplosionFactory* realFactory;
    unordered_map<std::string, Explosion> cache;

    string generateKey(ExplosionConfig& config)
    {
        return config.getType() + "_" +
            config.getColor() + "_" +
            to_string(config.getParticleCount()) + "_" +
            to_string(config.getSpread()) + "_" +
            to_string(config.getLifetime()) + "_" +
            to_string(config.getX()) + "_" +
            to_string(config.getY());
    }

public:
    ExplosionCacheProxy()
    {
        realFactory = new ExplosionFactory();
    }

    ~ExplosionCacheProxy()
    {
        delete realFactory;
    }

    Explosion create(ExplosionConfig &config) override
    {
        string key = generateKey(config);

        if (cache.find(key) == cache.end())
        {
            cout << "[Proxy] Cache MISS" << endl;
            cache[key] = realFactory->create(config);
        }
        else
        {
            cout << "[Proxy] Cache HIT" << endl;
        }

        return cache[key];
    }
};

```

ParticleDecorator.cpp

```
#pragma once
#include <iostream>

using namespace std;

class ParticlePrototype;

// =====
// IParticle
// Interface de base pour rendre les particules.
// =====
class IParticle
{
public:
    virtual void render() = 0;
    virtual ~IParticle() {}
};

// =====
// IParticleDecorator
// Classe de base pour les décorateurs de particules.
// Abstraite car le render n'a pas de comportement concret
// =====
class IParticleDecorator: public IParticle
{
protected:
    IParticle* decoratedParticle;
public:
    IParticleDecorator(IParticle* particle)
    {
        decoratedParticle = particle;
    }

    virtual void render() override
    {
        if (decoratedParticle)
        {
            decoratedParticle->render();
        }
    }

    virtual ~IParticleDecorator() { delete decoratedParticle; }
};

// =====
// Decorators concrets
// =====
class GlowDecorator : public IParticleDecorator
{
public:
```

```

    GlowDecorator(IParticle* particle) : IParticleDecorator(particle) {}

    void render() override
    {
        IParticleDecorator::render();
        cout << " -> Applying Glow Effect" << endl;
    }
};

class ShadowDecorator : public IParticleDecorator
{
public:
    ShadowDecorator(IParticle* particle) : IParticleDecorator(particle) {}

    void render() override
    {
        IParticleDecorator::render();
        cout << " -> Applying Shadow Effect" << endl;
    }
};

class FadeOutDecorator : public IParticleDecorator
{
public:
    FadeOutDecorator(IParticle* particle) : IParticleDecorator(particle) {}

    void render() override
    {
        IParticleDecorator::render();
        cout << " -> Applying FadeOut Effect" << endl;
    }
};

```